Raymond Bautista

Southern New Hampshire University (SNHU)

CS-360 Mobile Architecture & Programming

January 25, 2026

Project One: App Development Proposal – Event Tracking App

Introduction

In this project, as mobile developers at Mobile2App, we are developing an Android-based Event Tracking application (Option 2). This mobile application development effort spans multiple stages, including requirements analysis, design planning, UI/UX development, code implementation, and testing, all to be completed within an eight-week timeline. This document serves as the Application Development Proposal for the Events app, outlining the overall design strategy and initial implementation ideas for both the user interface and application architecture, aligned with the project's core goals and functional requirements.

Goals

The primary objective of this project is to provide a distraction-free personal hub that empowers users to master their daily schedules with minimal effort. At its core, the application focuses on reducing the mental burden of organization through an intelligent agenda system that automatically maps multi-day commitments across a chronological timeline, ensuring users have a complete, standalone snapshot of their daily bandwidth immediately. Development centers on creating a secure, private environment where personal data is safeguarded through authentication, while the interface prioritizes visual clarity by highlighting immediate priorities and providing proactive schedule reminders. By blending an intuitive event-creation process with

a transparent communication strategy regarding notifications, the project fulfills its mission of delivering a reliable and accessible tool that simplifies complex scheduling into a manageable and actionable personal event hub.

The main components and features of this app include:

- A database with two tables: one to store event details, and other for user credentials.
- A screen for login in or sign up in the app.
- A screen with a grid that displays all upcoming events.
- A screen that allows users to create new events, by entering time and general information in a form.
- A mechanism by which the user can remove events from the database.
- A mechanism by which the application will notify the user on the day that an event has been scheduled via SMS.

Users

This application is purposefully built for three primary user types: Professionals seeking to master high-stakes deadlines, Athletes maintaining strict training regimens, and Students juggling complex academic milestones. While their specific objectives range from corporate project management to physical consistency, these users share a universal goal of replacing mental clutter with a structured, reliable timeline. To achieve this, they require a secure, private repository that handles the "cognitive heavy lifting" of multi-day event repetition, paired with an intuitive interface that highlights immediate priorities through visual cues like theme-based color coding. By providing a "glanceable" dashboard and proactive SMS reminders, the app empowers

these diverse groups to transition from reactive stress to proactive control over their daily bandwidth.

UI Design

Screen and Features

Login and Register Screens

The entry experience is designed as a secure gateway, initially presenting a minimalist interface for existing users to authenticate via email and password. To minimize friction for new users, the screen is capable of expanding into a registration form that captures essential profile data, including a username and phone number. This purposeful data collection is the foundation for the app's notification ecosystem; by capturing the phone number during registration, the app establishes the necessary infrastructure to provide real-time event reminders via SMS, ensuring the user is connected to their schedule from their very first interaction.

Main Event Screen (AgendaView)

The Main Screen functions as a high-clarity dashboard designed to reduce cognitive load through a streamlined "Agenda View." This screen is organized as a chronological list where events are grouped under bold date headers in ascending order, with each item displaying the event name, description, and start/end hours. To eliminate the need for manual calculation, the interface automatically replicates multi-day entries across every date they span, providing a standalone snapshot of daily commitments. Visual hierarchy is reinforced by using the app's blue theme color to highlight "**Today**" and its specific **event times**, drawing the user's eye to immediate priorities and allowing for an instant grasp of their current bandwidth.

Add Event Form

The Add Event interface is a focused, linear form designed to transform abstract plans into concrete data with high accuracy. By utilizing structured fields for event titles and descriptions alongside integrated visual pickers, the screen guides the user through a reliable input journey. The intention here is to prevent "human error", such as typing invalid dates or times, by providing an intuitive, touch-based selection process. This ensures that every event saved to the database is technically valid and correctly formatted for the recurring logic of the main agenda.

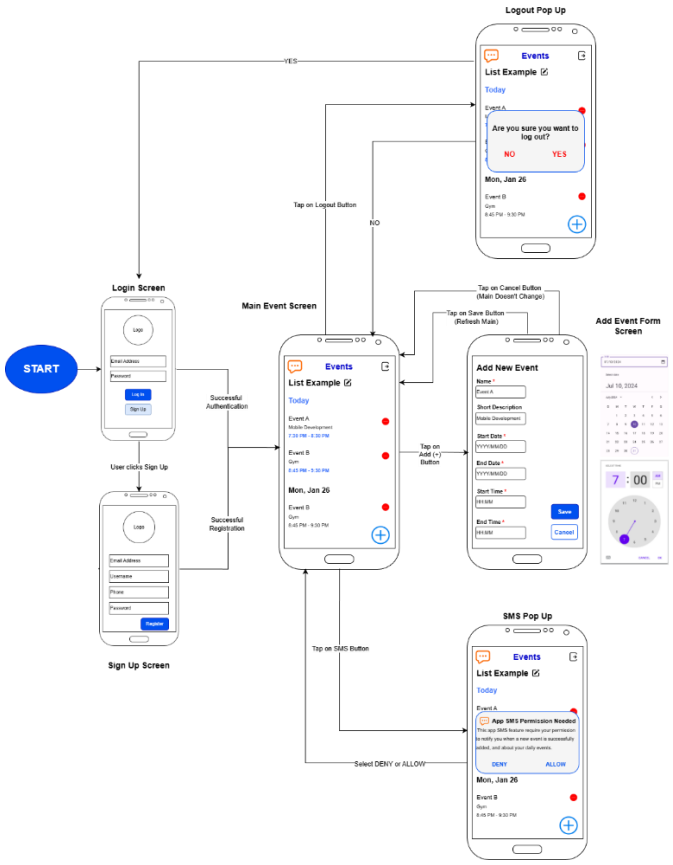
Logout and SMS Permission Dialogs

These dialogs act as critical touchpoints for user agency and informed consent. The logout prompt serves as a "confirmation bridge," ensuring that a user's session isn't terminated by an accidental tap, thus maintaining the integrity of their active workflow. Similarly, the SMS permission prompt utilizes a "Permission Priming" strategy by detailing the functional value of this feature, such as receiving daily schedule alerts, before the OS prompt appears. This transparent communication increases user trust and the likelihood that they will enable the features that make the app truly effective.

Screen Transition Flow

The application's navigation flow begins at the Login Screen, which serves as the primary gateway where users either authenticate existing credentials or navigate to the Sign Up Screen to register a new profile. Upon successful authentication or registration, the user is directed to the Main Screen, the application's central hub where the Agenda View dashboard allows for the viewing and deletion of events. From this dashboard, the flow branches into specialized tasks: the Floating Action Button transitions the user to the Add Event Form, returning them to a refreshed Main Screen upon saving, while the SMS Button launches a permission prompt that

returns to the dashboard regardless of the user's choice. To exit, the Logout Button triggers a confirmation prompt; if the user cancels or uses a back gesture, they are returned to their active schedule, whereas a "Yes" confirmation terminates the session and redirects the flow back to the initial Login Screen.



Code Design

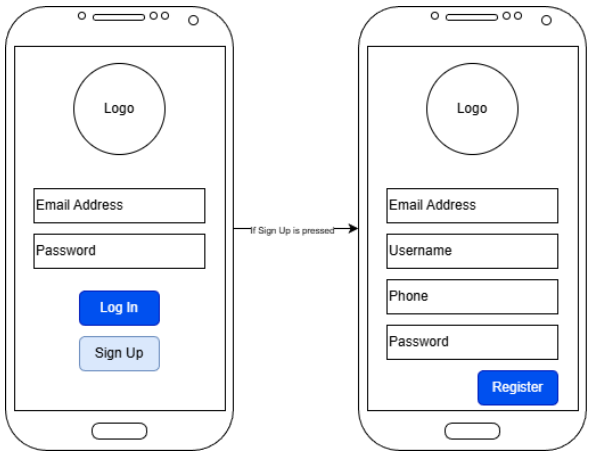
Firstly, this mobile application will be developed using the Java programming language, targeting a minimum Android SDK level of API 24 (Android 7.0, Nougat) to ensure broad backward compatibility across both modern and moderately older devices. This section outlines the core implementation details of the application, including the structure and functionality of the code and UI components for each screen, the selected architectural and data management approach, and the interaction flow between users, the application logic, and supporting system components.

UI Components

Login and Register Screens

The Login Screen serves as the entry point to the application and features a centered ImageView displaying the app logo, along with two TextInputLayout components for email and password entry. The password field is configured to mask input characters for security. In the Java layer, the Log In button's onClickListener triggers a validation routine that checks for empty fields and verifies user credentials. Upon successful authentication, an Intent grants the current session access to the event database and navigates the user to the main screen.

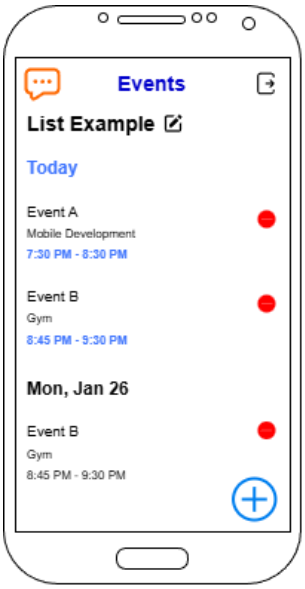
If the user does not yet have an account, a Sign Up option expands the screen to display additional TextInputLayout fields for username and phone number, enabling new user registration. From an architectural perspective, this screen interacts with the UserViewModel to insert a new user profile into the Room database. A key implementation detail is the use of a TextWatcher on the password field to provide real-time feedback on password strength or mismatches. The Cancel button simply invokes finish(), returning the user to the login view, while the Sign Up action performs the database insertion and automatically transitions the user to the newly created account's main screen.



Main Event Screen (AgendaView)

This screen represents the functional heart of the application. It features a TopAppBar displaying the app name and a logout ImageButton for session termination. Below the header, a RecyclerView serves as the primary content area, displaying event objects sorted in ascending order by time. The agenda logic is handled within the ViewModel, where the Java implementation retrieves a raw list of events from the database and transforms it into a list of DisplayItem objects. Each DisplayItem includes key attributes such as event name, short description, start date, end date, start time, and end time. Events occurring on the same day are grouped together under contextual headers (for example, “Today” or “Wed, Jan 28”), improving readability and chronological clarity. Each event row also includes a delete button, allowing users to remove events directly from the agenda.

An ExtendedFloatingActionButton (FAB) is anchored to the bottom-right corner of the screen, providing quick, thumb-reachable access to the primary action of adding new events. Additionally, the screen includes an SMS notification feature, where an ImageButton triggers a checkSelfPermission() call to verify SMS permissions. If permission is granted, a corresponding flag is stored in SharedPreferences; otherwise, the system prompts the user with the SMS permission dialog.

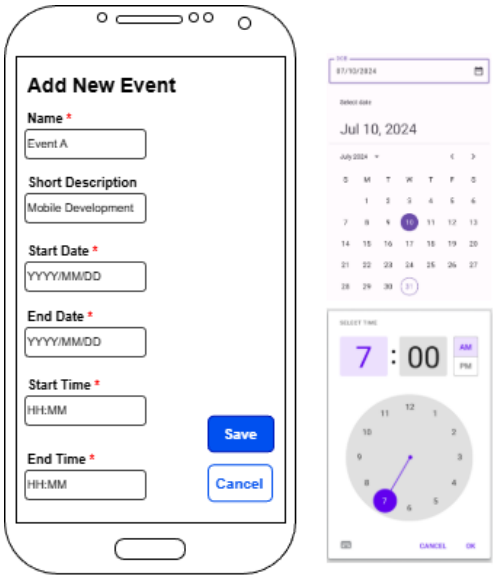


Add Event Form

This screen is implemented as a vertical form using a LinearLayout nested within a ScrollView to prevent content clipping on smaller devices. It includes EditText fields for the event name and description, while the date and time inputs are implemented as read-only TextInputLayout components. From a code perspective, setOnClickListener callbacks are attached to these fields to launch MaterialDatePicker and MaterialTimePicker components from the Material Design library, providing an intuitive and visually consistent calendar and clock-based selection experience for users (Google, n.d.). Once a date is selected, the Java SimpleDateFormat class converts the returned value into a human-readable string format (YYYY/MM/DD), which is displayed back in the corresponding field to provide clear visual confirmation of the user's input.

After the user completes all required fields and presses the "Save" button, the screen triggers the insertEvent() method in the ViewModel, encapsulating the input data into a new Event entity and persisting it in the Room database. Prior to insertion, the application validates the temporal logic by comparing Calendar objects for the start and end dates to ensure that the

end date does not precede the start date. Once validation succeeds, the database operation is executed on a background thread using AsyncTask or Executors, preserving UI responsiveness. Finally, the Activity calls finish() to return the user to the Main Screen, where the newly created event automatically appears in the agenda view through the established LiveData observer mechanism.

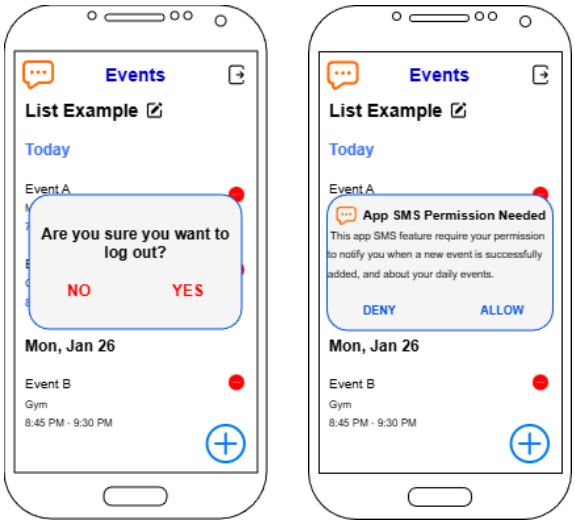


Logout and SMS Permission Dialogs

The Logout Prompt and SMS Permission Prompt are implemented using the MaterialAlertDialogBuilder class in Java. These prompts are not separate Activities or screens; instead, they function as modal dialogs that preserve the current Activity context, allowing users to confirm actions without disrupting their navigation flow. The Logout dialog uses setPositiveButton to clear the active user session and redirect the user to the Login screen, while setNegativeButton simply dismisses the dialog and returns the user to the current screen without any state changes.

For the SMS Permission Prompt, selecting the “Allow” action triggers Android’s standard requestPermissions() mechanism. Within the onRequestPermissionsResult callback, the

application captures the user's response and updates its behavior accordingly. If permission is granted, the SMS notification service logic is enabled; if denied, a boolean flag is stored to prevent repeated permission prompts, ensuring the app does not continue requesting access until the user explicitly re-enables the SMS feature by toggling the corresponding option again.



Data Flow and Method Architecture

I selected the Model–View–ViewModel (MVVM) architecture as the primary coding approach for this app because it enforces a clear separation of concerns, ensuring that the user interface remains lightweight while business logic is managed in a lifecycle-aware layer. By isolating event-tracking logic within a ViewModel, the application can preserve state across configuration changes such as screen rotations, preventing unnecessary data refetching when navigating between activities (Google, n.d.). This approach not only improves performance and responsiveness but also results in a more stable and predictable user experience.

To enable seamless data flow throughout the application, the app includes an AddEventActivity responsible for collecting user input through UI components such as DatePicker and TimePicker. When the user presses Save, the viewModel.insertEvent(event) call

packages the input into an Event entity and forwards it to a Repository layer. The repository acts as an abstraction between the UI and the Room database, encapsulating complex SQL operations and improving maintainability, testability, and scalability, especially for future integrations such as cloud synchronization services. To support a reactive and real-time user experience, the app leverages LiveData as the primary observation mechanism. The MainActivity observes changes in the event dataset, allowing the UI to update automatically whenever events are added or removed without requiring manual refresh logic (Google, n.d.). This reactive loop is particularly effective for Agenda View, as the ViewModel can process complex operations, such as expanding multi-day events into daily entries, on a background thread and delivering a finalized UI-ready dataset only when computation is complete.

Data Mapping: Input, Output, and Sources

Finally, this application utilizes a reactive data mapping workflow that seamlessly bridges user input with persistent storage and dynamic display. Primary data entry occurs within the Add Event form, where users provide textual details alongside date and time specifications facilitated by intuitive MaterialDatePicker and MaterialTimePicker components. Upon submission, this information is encapsulated into a Java object and persisted within the Room Database via the Repository. To complete the cycle, the Main Screen utilizes a LiveData observer that monitors the database for changes; as soon as the data is modified, the RecyclerView automatically re-renders the "Agenda View" to reflect the newly synced information. This lifecycle-aware architecture ensures that the data flow remains consistent, and the UI remains perfectly synchronized with the underlying database without requiring manual refreshes.

References

Google. (n.d.). *Date Pickers*. Retrieved from Material: https://m3.material.io/components/date-pickers/overview

Google. (n.d.). *LiveData Overview*. Retrieved from Android Developers: https://developer.android.com/topic/libraries/architecture/livedata

Google. (n.d.). *ViewModel Overview*. Retrieved from Android Developers: https://developer.android.com/topic/libraries/architecture/viewmodel